Idea Development Portfolio
EDUC 6144 Learning Sciences

Cogito

I think, therefore I learn.

An AI-Assisted Learning Environment
Built on the Six-Phase Cycle of Active Thinking

Mengchen Li
Penn Graduate School of Education
Instructor: Prof. Bodong Chen
Spring 2026

This document accompanies the Cogito interactive demo.
Demo: https://cogito-learn.vercel.app/

Abstract

This portfolio documents the project I designed and built this semester called Cogito, which is a conceptual prototype of an AI-assisted learning environment. The primary purpose of this record is to track the overall process of how my idea originated and how it gradually evolved throughout the semester's studies to eventually be presented in the form of a prototype. The motivation behind Cogito is that real understanding comes from active thinking, while AI tutoring often gives answers directly, which takes away the learner's own thinking. I believe that the learner's thinking should sit at the center.

Drawing on the learning sciences theories I learned this semester, together with my own understanding and experience of learning, I built the prototype around the following six-phase cycle: Orient, Predict, Investigate, Represent, Reflect, and Extend. The cycle is supported by an AI that switches among four modes based on a learner profile collected at the start of each session. The current demo uses a Python for loop as its example. The longer-term vision is a domain-agnostic system where learners can upload any material and receive a personalized learning path.

The portfolio has three parts. The first part consists of the topic presentation slides. The second part is the online demo that can be tried directly in a browser (***https://cogito-learn.vercel.app/***). The third part is this document, which records the full development process, including connections with course readings, the theoretical basis behind each design decision, the implementation issues I ran into, and my plans for the future.

Contents

Abstract 2

1. Problem Focus 4

2. Project Overview 4

   2.1 The role of the demo 4

   2.2 What is currently implemented 5

   2.3 What is not yet implemented 5

3. Development History 5

   3.1 Round one: the four-step prototype 5

   3.2 Round two: adding Orient and Extend 6

   3.3 Round three: refining the user experience 6

4. Connection to the Course 7

   4.1 Week-by-Week Influence 7

   4.2 Detailed Weekly Reflections 8

      4.2.1 Phase One. Finding Direction and Stance (Weeks 1–3) 8

      4.2.2 Phase Two. Core Architectural Formation (Weeks 4–6) 8

      4.2.3 Phase Three. Key Turning Point (Week 7) 11

4.2.4 Phase Four. Methodology, Stance, and Visual Layer (Weeks 8–14) 12
5. Theoretical Foundations 13
5.1 Productive Failure as Architectural Core 14
5.2 The DeFT Framework for Multiple Representations 14
6. The Six-Phase Cycle in Detail 14
6.1 Phase 1. Orient 14
6.2 Phase 2. Predict 15
6.3 Phase 3. Investigate 15
6.4 Phase 4. Represent 15
6.5 Phase 5. Reflect 15
6.6 Phase 6. Extend 16
7. Design Details and Visual Identity 16
7.1 The Name 16
7.2 The Logo 16
7.3 The AI Tutor Mascot 17
8. Rise-Above Synthesis 18
8.1 One Cycle, Multiple Theories 18
8.2 Integrated Architecture Diagram 18
9. Implementation Notes 20
9.1 Tech Stack 20
9.2 Orient Redesign 20
9.3 AI Tutor Costs 20
9.4 What Did Not Work 20
10. Future Vision 20
10.1 Subject-Agnostic Learning 21
10.2 Persistent Learner Profile 21
10.3 Auto-Generated Videos and Visualizations 21
10.4 Lightweight Community Layer 21
10.5 Teacher Dashboard 21
11. References 22
12. Appendices 23
Appendix A. How to Access the Demo 23
Appendix B. Design Decision Log 23

1. Problem Focus

My core question is how learners should interact with AI tools so that they actually learn something, instead of just getting an answer. From the learner's perspective, what should good AI-assisted learning look like?

This question comes from two places. One is personal experience. In my own studying and research I often go to some GenAI tools like ChatGPT, Claude, and Gemini for explanations, especially for fast clarifications of concepts or for help understanding articles. In the moment I feel I have understood, but over time I notice that some of my core abilities, such as the ability to explore, interpret, and dig into a problem, do not get exercised. Many concepts that I felt I had quickly grasped on the surface turn out, later on, to be ones I never really mastered.

The other place is my research experience. At the Penn AIED Lab I work on a project that uses small language models to support Socratic math tutoring. In this work I have noticed that the demands placed on AI models and AI prompts are actually very high. If the upstream design is weak, then once the Socratic tutoring conversation starts, the AI may ask the wrong kind of question, and the learning can drift. This made me feel that the overall design upstream of an AI tool, meaning how we design a structured learning framework, is critical to the actual learning that happens downstream.

In the topic presentation during this semester I also referred to Bastani et al. (2025), published in PNAS, which gave me a further push. They ran a controlled experiment in Turkey where high school students used GPT-4 to study math. The results showed that students using default GPT-4 without guardrails performed worse on later assessments than students who had not used AI at all. The reason is straightforward. AI is too quick to give answers. Students feel they are learning, while the actual cognitive work has been taken over by the AI. Once the AI is removed, the surface progress vanishes. The lesson I took from this study is that if AI removes the productive struggle that learning requires, it can in fact undermine learning. So my initial design idea became more specific: how can I make an AI learning tool that supports deep thinking instead of doing the thinking for the learner?

2. Project Overview

Based on my initial idea, I have evolved my thoughts into conceptual prototype called Cogito. Cogito is an AI-assisted learning environment whose central idea is that real understanding comes from active thinking. The name comes from René Descartes' Latin phrase Cogito, ergo sum, which translates as "I think, therefore I am." I rewrote it as the product slogan "I think, therefore I learn."

2.1 The role of the demo

The current demo focuses on one concept in Python programming. Once the learner enters the lesson, they go through six phases around that single concept. Each phase has a clear cognitive purpose.

The first phase is Orient. The learner answers six short questions covering background, prior experience, preferred explanation style, and what they usually do when they get confused. The system uses these answers to adjust the AI's tone and support style for the rest of the lesson.

The second phase is Predict. The learner sees a piece of code and must guess its output before any answer appears. They can choose from four options or write their own prediction. After they submit, the AI analyzes their prediction, and the system runs the code so that the learner can compare their thinking with the actual result.

The third phase is Investigate. The learner can modify the code in a sandbox, try four preset variants, run the result, and chat with an AI tutor. The AI responds in a way that fits the current learner, based on the learner profile from Orient.

The fourth phase is Represent. The learner can choose from seven different ways of understanding the same concept, including a line-by-line walkthrough, an animated trace table, a real-world analogy, a concept map, a with-versus-without comparison, and a video explanation.

The fifth phase is Reflect. The learner explains the concept in their own words. The AI reads the explanation and gives feedback that helps the learner notice where their explanation is still unclear.

The sixth phase is Extend. The learner faces a transfer challenge that asks them to use the same concept in a new context. After completion, the system shows where this concept sits in a larger course map and offers an optional learning journal.

2.2 What is currently implemented

The prototype now runs the full six phases end to end. The Orient phase contains six questions and supports custom input. The AI tutor uses the Claude API and adjusts its responses based on the learner profile. The Investigate sandbox provides four code variants. The Represent phase contains seven different representation types, including an imported video. The Reflect phase has AI feedback wired in. The Extend phase contains a transfer challenge, custom answers, a connection map, an optional journal, and a celebration animation at the end of the lesson.

2.3 What is not yet implemented

The current demo focuses on a single concept and a single lesson. It does not yet support uploading learning materials, multi-lesson paths across courses, or persistent learner profiles across sessions. These are discussed in Section 10, “Future Vision.”

3. Development History

Cogito did not start as a six-phase cycle. The first version was a four-step demo, and three rounds of major revision happened during the semester. I keep this development history here because one of the most valuable parts of this project is exactly how the idea kept becoming clearer through readings, class discussions, and peer feedback.

3.1 Round one: the four-step prototype

The first version of Cogito had four steps: Predict, Explore, Represent, and Reflect. I built this version after reading the productive failure literature in Week 4 and the multiple representations literature in Week 6. The core idea was that the learner should be required to make a guess before seeing the answer, because the act of guessing itself activates prior knowledge and creates the conditions for deep learning.

This four-step demo was already a working web application with a code editor, a simple AI tutor connected to the Claude API, and step-by-step navigation across the four phases. Users could submit predictions, run code, choose a representation style, and write reflections.

But in actual use the system showed several limitations. First, the system did not know who the learner was. Everyone got the same explanation in the same tone. I noticed this through borrowing from other strong examples I came across. For instance, I followed a domestic team's app called Hyperknow, where they build something like a learner profile early on, and where the system, by reading early signals from the learner, automatically guides the language style and tutoring style of the rest of the session. Putting these observations together, I realized that we can use different AI phrasing and interaction context for different learners, which gives more targeted learning. There was also no transfer challenge, so the lesson ended without checking whether the learner could use this concept somewhere new.

3.2 Round two: adding Orient and Extend

In Week 7, after reading the inquiry-based and problem-based learning literature, I expanded the cycle from four phases to six. The two new phases sit at the beginning and the end. Orient sits at the beginning and asks about the learner's background and preferences. Extend sits at the end and asks the learner to apply the concept in a new context.

The readings supported my decision to add these two phases. De Jong et al. (2023) showed that the timing of direct instruction matters and that some calibration of prior knowledge before inquiry is beneficial. The phrases also draw on inquiry-based learning and PBL cycles (Hmelo-Silver et al., 2018), where learners actively construct knowledge and iteratively refine their understanding through problem solving. And Hatano and Oura (2003) argued that the goal of education should be adaptive expertise, which emphasizes flexible application of knowledge.

Adding Orient also addressed the personalization problem. Once the system has collected six pieces of information from the learner at the start, the AI can change its tone and depth across the rest of the lesson. For example, a learner who prefers Socratic explanations will get questions in return. A learner who prefers direct explanations will get concise answers.

3.3 Round three: refining the user experience

The third round of changes came from peer feedback and from my own iteration on the demo. The most important refinements are below.

- The Orient phase changed from "all six questions on one page" to "one question per page" with smooth transitions. This made the Orient process feel more like a conversation than a form, and each question received more complete visual focus.
- The Investigate AI tutor's interaction was redesigned. It no longer occupies the main interface. It hides in a corner of the screen and floats up when needed. Visually, I used a small code-style face to suggest that it is a thinking partner, with no claim to authority over answers.
- The Reflect phase gained a three-part structure. The first part shows a quick comparison between the prediction and the actual output. The second part asks the learner to explain the concept in their own words and provides AI feedback on that explanation. The third part asks which representation was most helpful, building metacognitive awareness over time.

- A celebration animation was added at the end of the Extend phase. When the lesson ends, confetti, a congratulations title, and a warm encouragement message appear together, so the ending does not feel too cold.
- A video option was added to the Represent phase. Learners need richer modes, and from my own observation a video explanation can naturally complement text, diagrams, and animations.

4. Connection to the Course

4.1 Week-by-Week Influence

The table below summarizes which weeks contributed which ideas to Cogito. Detailed reflections follow.

Week	Topic	What it gave Cogito
Week 1	Learning Sciences Overview	Treating learning as a designed process, not a spontaneous event.
Week 2	Knowledge Building	The principle of improvable ideas. Every Cogito feature is an idea that can be improved.
Week 3	Cognition and Conceptual Change	The view that p-prims are resources, not obstacles. Cogito surfaces intuition in Predict.
Week 4	Motivation, Regulation, Failure	Productive failure as the core mechanism. Self-regulated learning as the foundation for the learner profile.
Week 5	Expertise and Adaptive Expertise	The goal of flexibly transferring routine procedures. This justifies the Extend phase.
Week 6	Multiple Representations	The seven-representation menu in the Represent phase. The DeFT framework.
Week 7	Inquiry and Problem Solving	The PBL tutorial cycle. The four AI modes. The timing of direct instruction.
Week 8	Analyzing Learning Processes	Verbal protocol analysis as a way to study learner thinking. It informed the Reflect phase.
Week 9	Collaboration, Argumentation, Scripting	Scripting as soft scaffolding. The phase navigation is a script.
Week 10	Learning in Networks and the Wild	The open-source value of transparency. The learner profile is fully visible.
Week 11	Design-Based Research	The iterative cycle that the whole project follows.
Week 12	Power and Politics	A reminder that AI in education is not neutral. Cogito centers learner agency.
Week 13	Learning by Making, Moving, Being	Constructionism and the value of learning through making. Cogito asks learners to construct their own explanations.

Week 14	Learning Sciences and Policy	The recognition that any AI learning tool eventually has to deal with policy at the school, district, and national levels. Cogito's API key model raises questions about who can access it.

Table 1. Weekly Cogito Project Influence Summary

4.2 Detailed Weekly Reflections

4.2.1 Phase One. Finding Direction and Stance (Weeks 1–3)

The first three weeks were a process of finding direction for me. My previous work at the AIED Lab was on concrete Socratic questioning design, with very micro-level concerns, for example, what kind of question could guide students to articulate the key step of a math concept on their own. This work gave me strong intuitions about how AI assists learning, but I did not have a clear picture of the theoretical foundation behind those intuitions. The Week 1 overview readings made me realize that learning sciences is a discipline with its own consensus and methodological tradition.

In Week 2 I read Scardamalia and Bereiter (2022) on knowledge building, and my core question was, if the best learning happens through participating in a community that collectively advances ideas, then what role does AI play in this process? A student sitting in front of a screen asking ChatGPT questions, the scene looks more like knowledge acquisition than knowledge building. This question shaped how I judged Cogito's boundaries. Cogito is currently an individual product. In future versions, I will look for ways to add a lightweight community layer where learners can see each other's predictions and reflections (see Section 10).

Week 3's influence on Cogito was much more direct. diSessa's (2022) discussion of p-prims changed the design of the Predict phase. Before reading this article, the prediction question I designed was a standard multiple-choice question with one correct answer and three wrong answers. At the time, I assumed that wrong answers were misconceptions to be corrected. After reading the article, I realized that this design would lose the most important thing. A student's prediction is actually very valuable, because it is the window into their current way of thinking. Every learner has many fragmented, context-dependent intuitions in their head, which are p-prims. These intuitions are material for constructing understanding. They should not be roughly cleared away. Based on this judgment, I made one very concrete adjustment. Beyond the four preset options, I added a free-text input. Learners can choose an option, or they can write down their prediction in their own words. If they choose free input, the AI analyzes the reasoning in their original words, first pointing out the reasonable parts, then helping them see the gap between the prediction and the actual result.

4.2.2 Phase Two. Core Architectural Formation (Weeks 4–6)

Weeks 4 through 6 were the key turning point of the whole project. The four papers I read in these three weeks each became one of the four cornerstones of Cogito's architecture.

Productive failure as architectural core (Week 4)

Kapur and Bielaczyc's (2012) paper on productive failure was the one I read most carefully this semester. Their research shows that students who first try to solve complex problems before receiving direct instruction often gain deeper understanding and better transfer ability. In the short term, the 'failure group' may underperform the direct instruction group, but in terms of long-term understanding and transfer, their learning is more solid. My understanding is that the

act of making a commitment first activates a student's existing knowledge and prepares the mind for the explanation that follows. This mechanism later became the architectural foundation of Cogito. Every phase is designed around active commitment.

Hanson's reflection on Wonderbits ran in the same direction as mine. His reading made the practical implications of PF clearer:

> PF allows students to first attempt and fail at complex problems without scaffolding, which actually builds richer cognitive structures for subsequent understanding. Although they may not solve problems successfully during the process, the greater number of representations and methods they generate leads to superior performance on post-tests and greater representational flexibility compared to direct instruction. It reminds me that instructional design should shift focus from 'avoiding mistakes' to 'making mistakes visible and integrating them': first provide space for exploration, then use teacher-led consolidation to organize students' emerging ideas into formal understanding.

What Hanson's words made clear to me is this: what matters in PF is making failure visible and integrating it. Letting students fail first is only one part. This is precisely why in Cogito, Predict must be immediately followed by Investigate and Reflect. Without a mechanism that lets failure be seen and discussed, PF degenerates into pure frustration.

Self-regulated learning and the four AI modes (Week 4)

Järvelä et al.'s (2018) article on self-regulated learning made me think from another angle. They emphasize that effective learning involves monitoring and regulating cognition, motivation, and emotion. They also distinguish self-regulation, co-regulation, and socially shared regulation. When I read this part, GenAI like ChatGPT came to mind naturally. When students learn with ChatGPT, who is regulating the learning process? In many cases it is the AI doing the regulating. The AI decides what to say, how to say it, and when to give examples, while the student only receives. This made me clearer about the role of AI in Cogito. It should support learners in regulating themselves. It should not regulate on their behalf. AI cannot take the entire learning process over.

This week also laid the foundation for the four AI modes that came later, namely Socratic Facilitator, Direct Instructor, Emotional Supporter, and Challenge Provocateur. Different students have different needs, and the support modes they are used to and the support modes they need may also be different. The Orient phase later asks learners 'what do you usually do first when you get stuck' precisely to give the AI some clues to help it judge when to use which mode of support.

I also wrote a line on Wonderbits, 'If struggle is good, why does struggle feel so bad?' This week did not fully answer that question, but it made me realize that what AI needs to guard is the boundary between 'productive struggle' and 'pure frustration'. This idea slowly turned into the design of the Reflect phase and the AI feedback inside it.

Adaptive expertise and the Extend phase (Week 5)

Hatano and Oura (2003) distinguish routine experts from adaptive experts. Routine experts are good at solving familiar problems with speed and accuracy. Adaptive experts can do this too, and they also know why a method works, and they can develop new methods when the context changes. If a learning tool ultimately just makes students better at getting answers, then what it trains is routine experts. Adaptive expertise is an important model of successful learning. This

point gave me the reason to add the Extend phase. The Extend phase asks learners to apply the same concept in a new context. Being able to transfer means understanding goes deeper. Being unable to transfer means understanding is still shallow.

Hatano and Oura also discuss how adaptive expertise develops. They argue that it is more likely to emerge in contexts where learners face varied and changing demands. Such participation often involves exploration and reflection, which support the flexible application of knowledge. These ideas later became my own design language. Every time I designed a phase, I would ask myself, is there variation here? Is there room for exploration? Is there space for reflection? Cogito's six-phase cycle is essentially about finding balance among these dimensions.

Multiple representations (Week 6)

Cogito's Represent phase currently has seven representations, line-by-line breakdown, trace table, animation, analogy, concept map, with-and-without comparison, and video. This design comes from Ainsworth's (2006) DeFT framework. DeFT stands for Design, Functions, and Tasks. Ainsworth argues that different representations differ in design features such as medium, level of abstraction, and dimensionality, and they also serve different functions, such as constraining misinterpretation, complementing each other, and promoting deeper understanding. Different learners and different tasks also suit different representations. She also reminds us that more representations are not always better. If a learner does not know how to make connections across different representations, the result may instead be cognitive load.

Jueran's comment on Wonderbits made me see the ethical meaning of multiple representations more clearly:

> Representations are ways things are revealed to us. In educational design, we need to keep in mind that different medium (graphs, media, language) all reveal knowledge differently to students. The ideal instructional design is combining different representations together to eliminate gaps and bias.

Jueran's comment lifts representations from the level of teaching method to the level of how knowledge is revealed. This made me realize that providing seven representations in the Represent phase serves two purposes. One is to help understanding. The other is to reduce the bias that comes from a single representation. If Cogito offered only one explanation style, it would invisibly filter out learners who are not moved by that style. This is also the reason I later insisted that the Represent phase should not pick a representation for the learner.

A thought Aashna posted on Wonderbits showed me that this principle can also be borrowed across contexts:

> This 'one-size-fits-all' framing is exactly what I needed for my problem. I kept describing the generic classroom problem from my own experience but couldn't name it precisely. A lesson plan is also a kind of representation and it needs to fit a specific classroom context the way a specific representation needs to fit a specific learner.

Aashna is working on AI-assisted teacher lesson planning, and she translated the principle of 'multiple representations should fit the learner' onto 'a lesson plan should fit the classroom context'. She showed me the generalizability of my design principle.

From this week I clarified two concrete design principles. First, the Represent phase should not decide for the learner which representation is best. It should let them choose for themselves. A logical learner may prefer the trace table, an intuitive learner may be moved more by analogy,

and a learner with strong spatial thinking may choose the concept map. Which representation a learner chooses is itself valuable information. Second, representations cannot keep growing without limit. My current design includes 7 of them. Every time I add one, I ask, is there an essential difference between it and the existing ones? The seven I finally chose try to span different dimensions, including linear and holistic, abstract and concrete, static and dynamic, visual and narrative.

4.2.3 Phase Three. Key Turning Point (Week 7)

The three readings in Week 7 played an important supplementary role for my project, and they were the turning point at which Cogito expanded from four phases to six phases in one step.

Hmelo-Silver et al. (2018) introduced the PBL tutorial cycle. They describe the PBL tutorial cycle as a loose script consisting of two broad components: problem exploration and knowledge integration. Problem exploration includes problem presentation, identifying important facts, brainstorming hypotheses or solutions, and identifying knowledge gaps. Knowledge integration includes self-directed study, returning to the problem, reflection, and assessment. When I used this cycle to look back at my original four-step design, I found that my 'Predict, Investigate, Represent, Reflect' could actually be seen as a simplified version of the PBL cycle, missing a beginning and an ending. So I added the two parts Orient and Extend, and the project expanded from four phases to six. The whiteboard in the PBL section of this article also inspired me. In a traditional PBL classroom, the group records facts, ideas, learning questions, and action plans on a whiteboard. The function of the whiteboard is to make the thinking process visible. Later I added an optional learning journal to the Extend phase, allowing learners to record questions they still have, connections they see, and things they want to come back to in the future.

de Jong et al.'s (2023) article on inquiry and direct instruction made me rethink when AI should intervene. They point out that direct instruction can occur before, during, or after inquiry. The optimal timing depends on the learner's prior knowledge and the complexity of the domain. This article directly influenced the Orient phase. Orient adjusts whether the system later provides a direct preview or a lighter hint, based on the learner's answers. The AI in the Investigate phase switching between different modes also comes from this line of thinking.

Linn et al. (2018) on knowledge integration also gave me inspiration. Instead of just telling students where they are wrong, AI feedback should help students refine their own explanations. I applied this to the Reflect phase. The original Reflect was scoring-style feedback. After the AI read the student's explanation, it would say 'mostly correct, but missed X'. After reading Linn, I shifted the Reflect phase away from presenting the score as the main feedback. Although automated scoring can support personalized guidance, the feedback itself should encourage learners to revisit evidence and revise the connections among their ideas. So right now, the AI does not score. Instead, it asks a follow-up question, guiding the learner to discover gaps in their own explanation. The learner does the integration work, and the AI only points out the gaps.

By the end of this week, Cogito had grown from four phases to six, AI had grown from a single mode to four modes, and Reflect had changed from scoring-style feedback to dialogic follow-up questioning. This was the week with the largest design change of the whole semester.

Aashna wrote this on Wonderbits:

> The more I think about principled AI use, the more I think it's not a new skill - it's applying existing skills to a new object. Teachers already evaluate curriculum kits and textbooks critically: whose learner is this designed for? What does this

assume? AI lesson plans need the same treatment. The difference is that AI outputs feel personalized in a way that makes critical reading harder.

Her core point is that 'principled AI use is not a new skill, it is applying existing critical skills to a new object'. This echoes Cogito's stance. Cogito's design philosophy is also about protecting the cognitive abilities learners already have, so that these abilities can still be activated when AI is present.

4.2.4 Phase Four. Methodology, Stance, and Visual Layer (Weeks 8–14)

Scripting and the architectural-layer perspective (Week 9)

Kollar et al.'s (2018) article on scripting made me re-understand Cogito's overall architecture. They argue that a script specifies and sequences collaboration phases, roles, and learning activities in order to guide the learning process. Cogito's six-phase navigation can be interpreted as a form of scripting, extending the idea from collaborative settings to an individual learning process. In the order of Predict, Investigate, Represent, Reflect, and Extend, each phase is preparing cognitively for the next phase. Predict gets the learner to put forward a mental model first, Investigate lets that model meet reality, Represent and Reflect help the model reorganize, and Extend tests whether this new model can transfer to a new context. This order itself carries a pedagogical function. This perspective also helped me explain Cogito more accurately. It is a structured cognitive process. It is not 'AI features plus several phases'. Later when I introduced Cogito to classmates, saying 'it is a script' conveyed my design intention better than saying 'it has six phases'.

DBR as a methodological framework (Week 11)

Fishman and Penuel's (2013) article on design-based implementation research provided a methodological framework for the whole project. They emphasize that DBR is a continuous cycle among design, deployment, observation, and revision. My whole semester of doing Cogito was essentially a DBR experiment. The three rounds of iteration correspond to three cycles of designing, deploying, and learning. The first round was the four-phase demo, the second round added Orient and Extend and expanded into six phases, and the third round polished UX, added confetti, video, and clearer Settings safety messages. Behind each round there are testable design conjectures. For example, the conjecture 'whether free-text prediction activates prior knowledge more than multiple choice' could be tested in the future through A/B testing.

Power, stance, and design choices (Week 12)

This week's content reminded me of an important issue. AI in education is never a neutral tool. My choice to use Anthropic's Claude rather than OpenAI's or Google's model is a choice. My specification in the prompt that the AI should not give answers directly is a choice. My making the learner profile fully transparent to the learner is also a choice. Behind each choice is my judgment about 'what learning is' and 'how much autonomy a learner should have'. These judgments need to be stated clearly.

Cogito has a stance. It refuses to give answers directly when the learner has not yet thought first. It refuses to store learner data on my server. It also refuses to decide what kind of help they need without asking the learner. These are all things this week helped me articulate.

Visual-layer upgrade (Week 13)

This week's theme was Papert's constructionism, and the broader tradition of 'learning by making'. Constructionism holds that learners form internal understanding by constructing external artifacts. This principle is very close to Cogito's Represent phase. Learners choosing

representations, writing their own explanations, and constructing their own examples are all processes of externalizing thinking.

Yiyi's topic presentation also influenced me a lot this week. Yiyi shared a math AI learning website she designed. She placed math concepts into a storified context, and every learning node was like a story node in an RPG game. She also designed a character called Sage, who gave students direction like a guide in a game, without giving the answer directly. The visual style of her website was also very complete, and the whole experience felt like a coherent world. After listening to her presentation, I realized that Cogito was still very thin at the visual and contextual layer. I had been mainly focused on functions before. Yiyi's presentation made me see that the texture layer is equally important. After that, I did several things. First, I upgraded Cogito from a functional demo into a more complete frontend product, redesigning the landing page to visually communicate Cogito's philosophy and the six-phase cycle. Second, I added a project introduction page, so that first-time visitors could quickly understand what Cogito is, who it serves, and what theoretical foundations sit behind it. Third, I added details with more warmth, such as the confetti animation after course completion, the code-style face on the AI mascot, and the color coding of different phases.

5. Theoretical Foundations

This section maps each design feature to its theoretical source.

Design feature in Cogito	**Theoretical source**
Six-question intake in Orient	Self-regulated learning (Järvelä et al., 2018). The timing of inquiry depends on regulating learner factors (de Jong et al., 2023).
Prediction before any answer	Productive failure (Kapur & Bielaczyc, 2012). P-prims as resources (diSessa, 2022).
Free-text prediction option	diSessa (2022) on the contextual richness of intuitive ideas. The multiple-choice format compresses this richness.
AI feedback on free-text predictions	Linn et al. (2018) on the principle that automated guidance and critique should refine, not replace, the learner's own ideas.
Sandbox in Investigate	Hmelo-Silver et al. (2018) on the inquiry component of PBL. De Jong et al. (2023) on inquiry-based instruction.
Four AI modes (Socratic, Direct, Emotional, Challenge)	De Jong et al. (2023) on personalized instruction. Hmelo-Silver et al. (2018) on expert PBL facilitation. Järvelä et al. (2018) on the regulation of cognition, motivation, and emotion.
Seven representation choices in Represent	Ainsworth's (2006) DeFT framework on multiple representations.
Construct-your-own-representation activity	Constructionism (Papert) and the self-explanation effect (Chi et al., 1989).

AI question after each reflection	Linn et al. (2018) on critique-and-refine dialogue. The goal is integration, not correction.
Transfer challenge in Extend	Hatano and Oura (2003) on adaptive expertise.
Connection map after Extend	Knowledge integration (Linn et al., 2018). The map shows the concept inside a network.
Visible and editable learner profile	Open educational systems (Santo, 2018).
Six-phase navigation as a script	Kollar et al. (2018) on scripting in CSCL. The script constrains interaction into a productive sequence.

Table 2. Theoretical basis for design features

Two readings deserve a longer discussion. They shaped Cogito at the foundational level.

5.1 Productive Failure as Architectural Core

Kapur and Bielaczyc (2012) argue that learners who struggle with a problem before receiving instruction develop deeper understanding than learners who receive instruction first. Their classroom studies show that even when the productive failure group performs worse on the immediate post-test, they outperform the direct instruction group on transfer assessments. This is the architectural foundation of Cogito. The whole six-phase cycle is a productive failure machine. Predict forces learners to commit before knowing the answer, Investigate gives them a way to test their commitment, Represent and Reflect process the gap between prediction and reality, and Extend tests whether the resulting understanding holds in a new context. Bastani et al.'s (2025) study on AI tutoring without guardrails is reverse evidence for the same point. When AI removes the productive failure step by giving answers immediately, learning collapses.

5.2 The DeFT Framework for Multiple Representations

Ainsworth (2006) proposed the DeFT framework, which includes the three dimensions of Design, Functions, and Tasks. Different representations differ in design features such as medium, level of abstraction, and dimensionality. They serve different functions, such as constraining interpretation, complementing each other, and constructing deeper understanding. The right representation depends on the learner, the topic, and the task. Cogito's Represent phase is a direct application of this framework. The seven representation options span different design dimensions. Line-by-line explanation is concrete and sequential, the trace table is abstract and tabular, the animation is visual and temporal, the analogy is familiar and conceptual, the concept map is spatial and relational, the comparison is contrastive, and the video is narrative. Each one highlights a different aspect, and the learner can pick the one they need.

6. The Six-Phase Cycle in Detail

This section describes each phase from the learner's perspective, the cognitive purpose behind it, and the specific design decisions that distinguish it from a generic learning app.

6.1 Phase 1. Orient

Purpose. Activate prior knowledge, calibrate the AI to the learner, and frame the upcoming work.

What the learner does. Six short questions appear one at a time, with smooth transitions. The questions cover programming background, languages used, learning purpose, preferred explanation style, approach to confusion, and behavior when stuck. Each question has prepared options and a free-text custom field. The learner can use either.

Why this design. The one-question-per-page flow gives each question full visual attention. The custom field respects learners whose answers do not fit any prepared option. Together the six questions construct a learner profile, which the AI uses for the rest of the lesson. The profile is fully visible to the learner and editable.

6.2 Phase 2. Predict

Purpose. Force the learner to commit to a mental model before any answer appears. This is the productive failure moment.

What the learner does. The learner sees a Python code snippet that creates a list of fruits and iterates over it. They choose from four prepared predictions or write their own. Once they submit, the system runs the code, shows the output, and provides AI feedback on their prediction.

Why this design. The free-text option matters because it lets the learner express their actual mental model. The AI feedback is calibrated to validate the reasoning behind a wrong prediction before correcting the outcome.

6.3 Phase 3. Investigate

Purpose. Let the learner test their understanding by manipulating code and observing results.

What the learner does. The learner can run four prepared variations of the original code. With index, skipping cherry, reverse order, and the same as the original. They can also chat with an AI tutor that floats in from the corner of the screen. The tutor responds in the style chosen during Orient.

Why this design. Hands-on manipulation makes inquiry feel like inquiry. The four prepared variations give the learner a structured starting point so they do not freeze in front of an empty editor. The AI tutor provides soft scaffolding only when asked, so the learner stays in the lead.

6.4 Phase 4. Represent

Purpose. Construct a representation of the concept that fits how the learner thinks.

What the learner does. The learner chooses from seven representation options. Line-by-line breakdown, trace table, animation, real-life analogy, concept map, with-and-without comparison, and video walkthrough. They can try as many as they want.

Why this design. Ainsworth (2006) shows that no single representation works for all learners. Cogito refuses to choose one for the learner. The learner can browse multiple representations and compare which one makes the concept resonate. This metacognitive comparison is itself a learning event.

6.5 Phase 5. Reflect

Purpose. Consolidate learning by translating it into the learner's own language and surfacing gaps.

What the learner does. Three sub-activities. First, the system shows a summary of the learner's prediction versus actual output. Second, the learner writes an explanation of the for loop concept

in their own words, and receives an AI follow-up question that probes a gap. Third, the learner tags which representation was most helpful for them.

Why this design. A single follow-up question respects the learner's authorship over their own explanation. The final metacognitive tag builds long-term awareness of how the learner learns best.

6.6 Phase 6. Extend

Purpose. Test transfer and embed the concept in a larger knowledge map.

What the learner does. The learner faces a transfer challenge with a new code snippet that uses range instead of a list. They choose or write a prediction, run the code, and receive feedback. After the challenge, they see a connection map placing the current concept among related concepts. They can write an optional journal entry. When they complete the lesson, a celebration animation plays.

Why this design. Hatano and Oura (2003)'s ideas informed the design of the Extend phase, which asks learners to apply concepts in new contexts as a way to probe the depth and flexibility of their understanding. The connection map shows that learning continues beyond this lesson. The celebration animation responds to my peer's feedback that the original ending was too cold. Real learning deserves real moments of joy.

7. Design Details and Visual Identity

7.1 The Name

Cogito comes from René Descartes' Latin phrase, Cogito, ergo sum. I think, therefore I am. The product slogan is 'I think, therefore I learn'. This single-line transformation captures the central design principle of the product. Real learning happens through active thinking, not through passive consumption.

The name is also short and memorable, and immediately signals a focus on cognition. Most Chinese-speaking learners I tested with recognized it immediately, since Descartes' phrase is well known.

7.2 The Logo



Figure 1. The Logo of Cogito

The logo is a wordmark composed of three simple geometric elements. A large C drawn as an open arc. A smaller O drawn as a complete circle floating above and to the right. A small solid dot between the C and the O. Together with the wordmark gito, they spell Cogito.

The logo carries five layers of meaning:

Layer 1. Letterform identity. C and O are the first two letters of Cogito. The graphic and the wordmark form a continuous object.

Layer 2. Thought bubble. The small O floating above the C is like a thought bubble rising from a mind. The dot between them is the connecting bubble that appears in comic-style thought balloons. At first glance, the mark looks like a mind with an idea.

Layer 3. The learning process. The C represents the learner, open and ready to receive new understanding. The O represents the moment of insight or the new concept that emerges. The dot represents the bridge moment of effort transforming into understanding.

Layer 4. Openness and continuity. The C is deliberately drawn as an open arc that never closes into a complete circle. This openness conveys a message about learning itself. Learning is never fully finished. The C stays open because the learner is always ready for the next cycle.

Layer 5. Simplicity as clarity. The logo uses only three geometric elements, with no decoration. This simplicity reflects the product philosophy. Cogito reduces the learning process to its essential steps. The shape of the logo does the same.

7.3 The AI Tutor Mascot

The AI tutor in the Investigate phase is shown as a small floating button with a code-style face. The face is the text { > _ ° } drawn in a monospaced font. This is intentional. The face looks like a friendly thinking partner. When the AI is generating a response, the face changes to { - _ - }, which signals concentration. When the chat is empty, the face becomes { °_° }, which signals attentiveness. These small changes make the interface feel alive without falling into anthropomorphism.

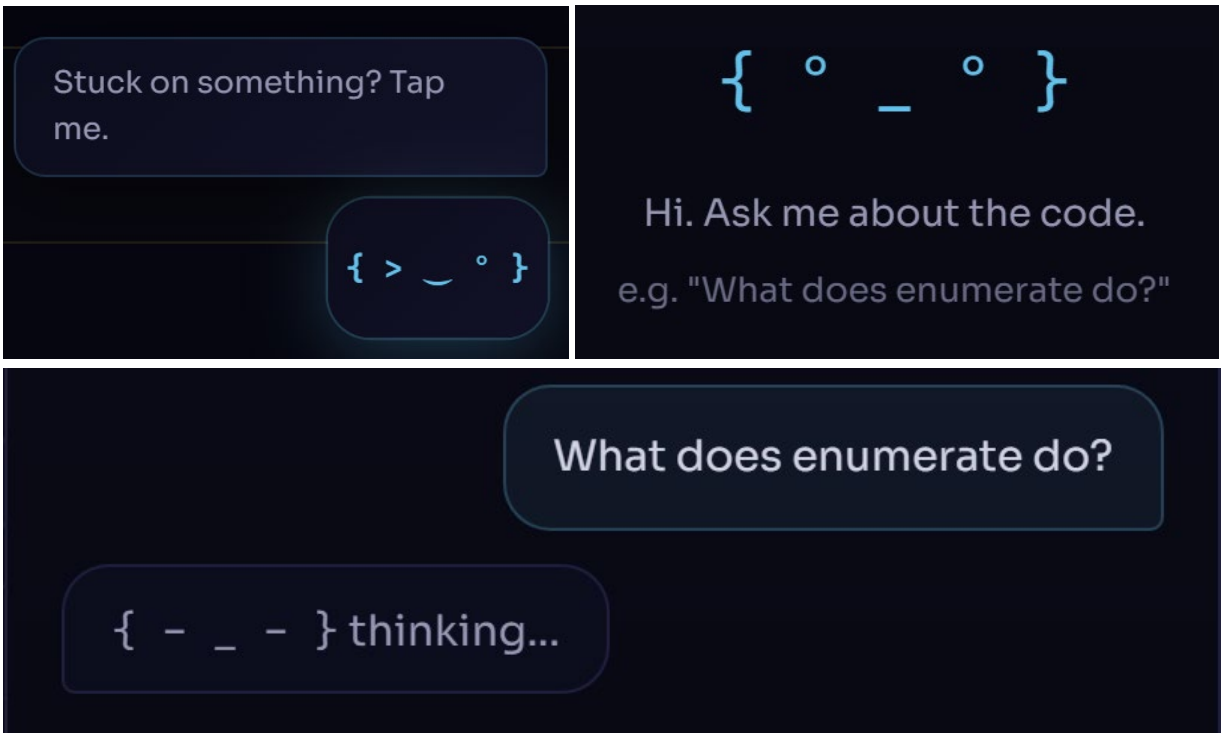


Figure 2. The AI tutor mascot.

8. Rise-Above Synthesis

I want to put several core concepts that recurred across different weeks this semester into one architectural diagram, see clearly the relationships among them, and then explain how this relational map supports Cogito's overall design.

8.1 One Cycle, Multiple Theories

Every theory I read in this course I initially understood separately. Productive Failure is Kapur and Bielaczyc's research on 'try first, then explain'. Knowledge Integration is Linn's framework on how learners construct understanding through comparison and refinement. Self-Regulated Learning is Järvelä's account of how learners regulate their own cognition, motivation, and emotion. Adaptive Expertise is Hatano and Oura's concept of flexible transfer. Multiple Representations is Ainsworth's DeFT framework. These theories, while originating from different traditions, can be interpreted as describing complementary aspects of a broader learning process. This seeing emerged slowly. It surfaced layer by layer through the process of repeated reading, repeated designing, and repeated cross-referencing with my own Cogito. When I put the six-phase design diagram and these theories on the same whiteboard, the cycle itself revealed its shape.

The cycle goes roughly like this: The learner is activated from some state, generates a tentative understanding of a new concept, takes that understanding into reality to test it, encounters the gap between the model and reality, reorganizes the understanding around that gap, and finally tests in a new context whether this new understanding holds. Activation, generation, encounter, refinement, transfer. This is a complete cycle, and it does not stop here. Every time it finishes, it triggers the next one, because a new transfer is itself the activation of the next concept.

Each theory I learned in class is actually illuminating one moment in this cycle. Productive Failure focuses on the moment of 'generation', telling us that committing to a wrong prediction drives learning more than passively receiving a correct explanation. Knowledge Integration focuses on the moment of 'refinement', telling us that good feedback should let the learner reorganize the explanation themselves. Adaptive Expertise focuses on the moment of 'transfer', telling us that real understanding must be tested in a new context. SRL is the meta-awareness that runs through the whole cycle. Scripting is the structure that strings these five moments into a coherent process.

8.2 Integrated Architecture Diagram

The diagram below draws out the relationships I just described. It is divided into four layers. The meta layer at the top, the theoretical anchors and core cycle in the middle, and the support layer at the bottom.

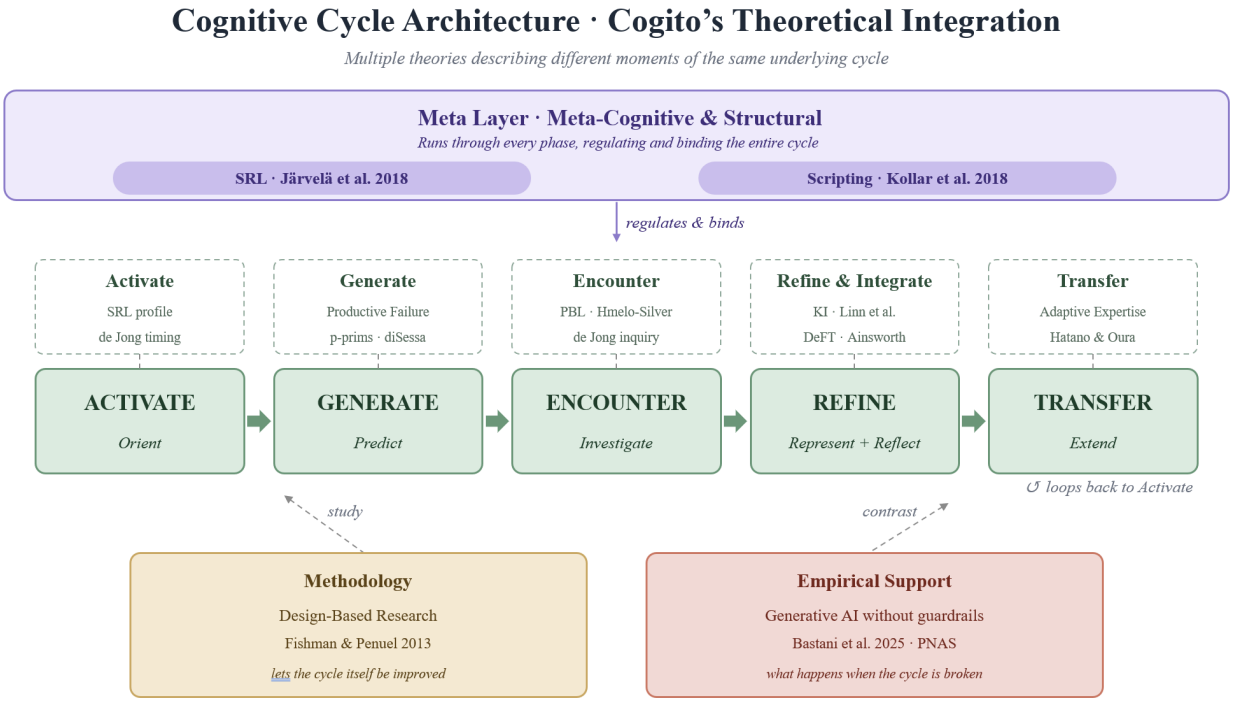


Figure 3. Cogito's cognitive cycle architecture.

The meta layer consists of SRL and Scripting. Järvelä et al.'s (2018) SRL tells us that learners need to monitor and regulate their own cognition, motivation, and emotion. This monitoring runs above and through all phases. Kollar et al.'s (2018) Scripting is a structural constraint that binds multiple scattered cognitive activities into a process with a coherent order. Together these two form the meta layer above the cycle.

The core cycle has five phases. Above each phase is a theoretical anchor that names the theory primarily illuminating that phase. The activation phase corresponds to SRL's prior calibration and de Jong et al.'s (2023) account of the timing of direct instruction. The generation phase corresponds to Kapur and Bielaczyc's (2012) productive failure and diSessa's (2022) concept of p-prims. The encountering-the-gap phase aligns with aspects of the PBL tutorial cycle, particularly the identification of knowledge gaps and de Jong's account of inquiry-based instruction. The refinement phase corresponds to Linn et al.'s (2018) Knowledge Integration and Ainsworth's (2006) DeFT framework. The transfer phase corresponds to Hatano and Oura's (2003) concept of adaptive expertise.

At the bottom are two project-level supports. The first is the methodology. Fishman and Penuel's (2013) design-based research lets Cogito's overall design be continuously improved through use. My three rounds of iteration on Cogito over the whole semester were a small DBR practice. Each round proposed new design conjectures based on the observations from the previous round, and tested them in the next version. The second is empirical support. Bastani et al. (2025)'s study responds at the level of empirical data to Cogito's core stance, that cognition only has meaning when the learner completes it themselves.

9. Implementation Notes

This section documents the practical realities of building Cogito. The challenges, the workarounds, and what I learned from them.

9.1 Tech Stack

Cogito is built as a single-page React application using Vite as the build tool. It is deployed to Vercel from a GitHub repository. The AI features call the Anthropic Claude API directly from the browser, with the API key stored in the user's localStorage. There is no backend server, because the design intentionally keeps user data on the user's machine.

9.2 Orient Redesign

The original Orient phase was a single page with all six questions visible at once. User testing with people I invited showed that this looked overwhelming and that users skipped through questions without thinking. The fix was to split the questions into six pages with smooth animated transitions between them. Each page shows one question with full visual focus. The animation gives the learner a brief pause to think before answering. This change took longer than expected, because the React state needed to track answers across all six pages while still allowing back-and-forth navigation. The final solution uses a single state object keyed by question, and each page renders one slice of that state. Going back to a previous question shows the previous answer.

9.3 AI Tutor Costs

Calling the Anthropic Claude API for every chat message adds up. During development, I made several decisions to keep costs manageable. The system prompt is sent once per conversation, not once per message. The maximum response length is capped at 800 tokens. The model used is Claude Sonnet 4, which is a balance of cost and quality.

9.4 What Did Not Work

Two ideas were tried and removed.

A confidence slider in Predict. The original Predict phase had a slider asking 'how confident are you' on a scale from wild guess to very confident. User testing showed that learners filled it in mechanically. They were not actually monitoring their own confidence. The slider added complexity without adding value, so I removed it.

Multi-provider AI support. I considered letting users connect to OpenAI, Gemini, or Anthropic APIs. The implementation was painful, because each provider had a different API format, and the testing surface tripled. I judged that one fully reliable provider was better than three with intermittent bugs. Cogito currently uses Anthropic Claude only.

10. Future Vision

The current demo is a focused proof of concept. It teaches one Python concept through one full cycle. The future vision is much larger. This section describes what Cogito should become.

10.1 Subject-Agnostic Learning

The biggest leap is making Cogito work for any subject. The six-phase cycle is already domain-general at the level of cognitive process. What needs to be added is the ability to take a piece of source material and automatically generate the content for each phase.

The flow would be like this: a learner uploads a textbook chapter, a research paper, a Wikipedia article, a piece of sheet music, or a recipe. The Cogito AI reads the material and generates the following items in order. A short Orient introduction with prior-knowledge questions specific to the topic. A predict-friendly scenario where the learner can make a guess. A sandbox or interactive environment for the Investigate phase. Several representation options for the Represent phase. Reflection prompts that match the topic. A transfer challenge that uses the same concept in a new context.

Building this requires three large components. A content analyzer that extracts key concepts from source material. A content generator that creates predictions, sandboxes, representations, and reflection prompts. An orchestrator that decides which AI mode and pacing fit the current learner and the current topic.

10.2 Persistent Learner Profile

The learner profile in the current demo only exists for one session. A future Cogito should have a profile that persists across sessions. Over time, the profile would learn that the user prefers metaphors when learning humanities topics and trace tables when learning programming topics. The AI would adjust accordingly.

A persistent profile would also enable a long-term concept map. As the learner studies more topics, the system could show how concepts connect across domains. A user studying biology and music theory would see that the concept of pattern recognition appears in both.

10.3 Auto-Generated Videos and Visualizations

The video walkthrough in the current demo is a manually uploaded file. A future Cogito should auto-generate explanation videos using a pipeline that combines text-to-speech narration with animated representations of the concept. NotebookLM-style audio explanations and AI-generated diagrams could fill in the representations that I cannot manually create for every concept.

10.4 Lightweight Community Layer

Cogito is currently a single-player experience. A future version could add a lightweight community layer. After a user submits their own prediction, anonymized predictions from other learners could appear. The user could see what percentage of learners predicted the same output. This would create a sense of shared inquiry without requiring synchronous collaboration. A discussion thread could appear at the end of each lesson, where learners exchange the representations they constructed.

10.5 Teacher Dashboard

If Cogito is used in a classroom, teachers will need a way to see how students are progressing. A teacher dashboard would show which students have completed which lessons, where students are getting stuck, and which representations they chose. The dashboard would not collect personal data beyond what is needed to support the teacher's instructional decisions.

11. References

Ainsworth, S. (2006). DeFT: A conceptual framework for considering learning with multiple representations. *Learning and Instruction, 16*(3), 183–198. https://doi.org/10.1016/j.learninstruc.2006.03.001

Bastani, H., Bastani, O., Sungu, A., Ge, H., Kabakcı, Ö., & Mariman, R. (2025). Generative AI without guardrails can harm learning: Evidence from high school mathematics. *Proceedings of the National Academy of Sciences, 122*(26). https://doi.org/10.1073/pnas.2422633122

Chi, M. T. H., Bassok, M., Lewis, M. W., Reimann, P., & Glaser, R. (1989). Self-explanations: How students study and use examples in learning to solve problems. *Cognitive Science, 13*(2), 145–182. https://doi.org/10.1207/s15516709cog1302_1

de Jong, T., Lazonder, A. W., Chinn, C. A., Fischer, F., Gobert, J. D., Hmelo-Silver, C. E., et al. (2023). Let's talk evidence—The case for combining inquiry-based and direct instruction. *Educational Research Review, 39*, 100536. https://doi.org/10.1016/j.edurev.2023.100536

diSessa, A. A. (2022). A history of conceptual change research: Threads and fault lines. In R. K. Sawyer (Ed.), *The Cambridge handbook of the learning sciences* (3rd ed., pp. 114–133). Cambridge University Press. https://doi.org/10.1017/9781108888295.008

Fishman, B., Penuel, W. R., Allen, A., & Cheng, B. H. (2013). Design-based implementation research. *Yearbook of the National Society for the Study of Education, 112*(2). Teachers College Record.

Hatano, G., & Oura, Y. (2003). Commentary: Reconceptualizing school learning using insight from expertise research. *Educational Researcher, 32*(8), 26–29.

Hmelo-Silver, C. E., Kapur, M., & Mastra, M. (2018). Learning through problem solving. In F. Fischer, C. E. Hmelo-Silver, S. R. Goldman, & P. Reimann (Eds.), *The international handbook of the learning sciences* (pp. 210–220). Routledge.

Järvelä, S., Hadwin, A. F., Malmberg, J., & Miller, M. (2018). Contemporary perspectives of regulated learning in collaboration. In F. Fischer, C. E. Hmelo-Silver, S. R. Goldman, & P. Reimann (Eds.), *The international handbook of the learning sciences*. Routledge.

Kapur, M., & Bielaczyc, K. (2012). Designing for productive failure. *Journal of the Learning Sciences, 21*(1), 45–83. https://doi.org/10.1080/10508406.2011.591717

Kollar, I., Wecker, C., & Fischer, F. (2018). Scaffolding and scripting (computer-supported collaborative learning). In F. Fischer, C. E. Hmelo-Silver, S. R. Goldman, & P. Reimann (Eds.), *The international handbook of the learning sciences* (pp. 340–350). Routledge.

Linn, M. C., Gerard, L., Matuk, C., & McElhaney, K. W. (2018). Inquiry learning and opportunities for technology. In F. Fischer, C. E. Hmelo-Silver, S. R. Goldman, & P. Reimann (Eds.), *The international handbook of the learning sciences* (pp. 221–233). Routledge.

Santo, R. (2018). Open source culture as inspiration for design and educator learning networks. In S. Yoon & S. Baker-Doyle (Eds.), *Networked by design*. Routledge.

Scardamalia, M., & Bereiter, C. (2022). Knowledge building and knowledge creation. In R. K. Sawyer (Ed.), *The Cambridge handbook of the learning sciences* (3rd ed., pp. 385–405). Cambridge University Press. https://doi.org/10.1017/9781108888295.024

12. Appendices

Appendix A. How to Access the Demo

The Cogito demo is deployed on Vercel. The link can be accessed through the URL printed on the cover page (***https://cogito-learn.vercel.app/***). To use the AI features, click the gear icon in the top right corner of the demo home page, and enter an Anthropic Claude API key. The key is stored in the user's browser localStorage and is only sent to the Anthropic API. It never reaches my server, because there is no server.

Without an API key, the demo can still run end to end. The static parts of each phase work without AI. Only the AI feedback in Predict and Reflect, and the chat with the AI tutor in Investigate, require a key.

Appendix B. Design Decision Log

- Round 1. Built a four-step demo with Predict, Investigate, Represent, and Reflect.
- Round 2. After Week 7, expanded to six phases by adding Orient and Extend.
- Round 2. Added four AI modes.
- Round 2. Added seven representation options.
- Round 3. Replaced the multi-question Orient page with a one-question-per-page flow.
- Round 3. Added the celebration animation in the Extend phase.
- Round 3. Added video as the seventh representation option.
- Round 3. Strengthened the API key security messaging in Settings.
- Round 3. Split the codebase into App.jsx and Demo.jsx for maintainability.
- Round 3. Built the landing page with five sections matching the headings. Why, the Cycle, Inside the Session, Theory, and Audience.

AI Use Statement

I used Claude and Typeless for translation and Grammarly for grammar and phrasing. All ideas and reflections are my own.